

MINOR ASSIGNMENT-07

Inter-Process Communication using Pipes & FIFO

Practical Programming with C (CSE 3544)

Publish on: 14-12-2024

Course Outcome: CO₄

Program Outcome: PO₃

Submission on: 21-12-2024

Learning Level: L₅

Problem Statement:

Experiment with inter-process communication mechanism(IPC) using pipes and FIFOs(named pipes).

Answer the followings:

1. Determine the number of file descriptors (fds) will be opened for the program that becomes process.
Write theirfd numbers.

```
int main(void) {  
    fprintf(stderr, "ITER\n");  
    while(1);  
    return 0;  
}
```

of FDs and their number

--	--

2. State the number of FDs will be opened for each of the process.

```
int main(void) {  
    fork();  
    fork();  
    fprintf(stderr, "hello\n");  
    return 0;  
}
```

of processes and FDs for each process

--	--

3. Check the directory; **\$ls /proc/PID/fd** to find the number of FDs the following program generates when it becomes process. Assume **read.c** is an existing file in your PWD.

```
int main() {  
    int fd,i;  
    for(i=0;i<10;i++) {  
        fd=open("read.c",O_RDONLY);  
        if(fd==-1) {  
            perror("Open error");  
            return 1;  
        }  
        sleep(2);  
        printf("FD Number=%d\n", fd);  
    }  
    return 0;  
}
```

FDs opened and output at the printf line

--	--

4. Check the directory; **\$ls /proc/PID/fd** to find the number of FDs the following program generates when it becomes process. Assume **read.c** is an existing file in your PWD.

```
int main() {
    int fd, i;
    for(i=0; i<10; i++) {
        fd=open("read.c", O_RDONLY);
        if(fd%2==0)
            printf("%d..\n", fd);
    }
    sleep(2);
    return 0;
}
```

FDs opened and output at the printf line

--	--

5. Study the use of **read()** and **write()** system calls for unformatted I/O.

```
#define BLKSIZE 8
int main(void) {
    char buf[BLKSIZE];
    read(STDIN_FILENO, buf, BLKSIZE);
    write(STDOUT_FILENO, buf, BLKSIZE);
    return 0;
}
```

Input to the program

- (a) STUDENT [Exactly 8 bytes including enter]
- (b) STUDENTS [Exactly 9 bytes including enter]
- (c) STUDENTSpwd [More than 8 byte]
- (d) STUDENTSpwd;who [More than 8 byte]
- (e) STUDENTSecho \$\$ [More than 8 byte]
- (f) STUDENTSBETCH [More than 8 byte]
- (g) studentsecho "USP IS TO PRACTICE" [More than 8 byte]
- (h) STUD [Less than 8 byte]

Observe the corresponding output

--

6. Write the number of file descriptors will be opened for the following code snippet. Verify the descriptor numbers by exploring the **fd** folder for the process in the directory **/proc/PID**.

```
#include<stdio.h>
#include<unistd.h>
#include<errno.h>
int main(void)
{
    int fd[2], fs[2], fds[2];
    pipe(fd);
    pipe(fs);
    pipe(fds);
    return 0;
}
```

of Processes& FDs with number

--	--

7. State the number of file descriptors will be opened for the below given code. Can you able to show the file descriptors in your machine? [Y — N]

```
#include<stdio.h>
#include<unistd.h>
#include<errno.h>
int main(void)
{
    int fd[2],fs[2],fds[2];
    if(pipe(fd) == -1){
        perror("Failed to create the pipe");
        return 1;
    }
    if(pipe(fs) == -1){
        perror("Failed to create the pipe");
        return 2;
    }
    if(pipe(fds) == -1){
        perror("Failed to create the pipe");
        return 3;
    }
    return 0;
}
```

of Processes & FDs with numbers

8. Write the descriptor numbers attached to both parent and child process file descriptor table(PFDT). Verify the descriptor numbers by exploring the **fd** folder for the process in the directory **proc**.

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(void) {
    int fd[2],fs[2],fds[2];
    pid_t pid;
    pipe(fd);
    pid=fork();
    if(pid==0){
        pipe(fs);
        pipe(fds);
    }
    else{
        wait(NULL);
        printf("Parent waits\n");
    }
    return 0;
}
```

Parent Process FDs	Child Process FDs

9. Write the descriptor numbers attached to both parent and child process file descriptor table(PFDT). Verify the descriptor numbers by exploring the **fd** folder for the process in the directory **proc**.

```
int main(void){int fd[2],fs[2],fds[2];
    pipe(fd);
    pid_t pid=fork();
    if(pid!=0){
        pipe(fs);
        pipe(fds);
    }
    else{
        wait(NULL);
        printf("Parent waits\n");
    }return 0;}
```

Parent Process FDs	Child Process FDs

10. Develop a program to write and read a message using **pipe()** for a single process. (*Hint: No need to use **fork()** and the main process will create and implement the pipe for both writing and reading.*)

Code here

Specify: Input & output

11. Here, use the **fork()** system call to create a child process. The child process will write a message into the pipe and the parent process will read the message from the pipe. The parent process will display the message on *stdout*. Design a program to establish the communication using pipe between the processes.

Code here

Specify: input & output

12. **dup()** and **dup2()** are used to duplicate the file descriptor table of any process. **dup()** is taking one argument and duplicate that file descriptor in first vacant descriptor of the table, where as **dup2()** is taking two arguments and coping the first descriptor in the second argument. Check the output of the following program and follow the instruction given in comment line to observe fd table of the corresponding process.

```
int main()
{
    int fd1,fd2,n;
    printf("PID=%ld\n", (long)getpid());
    fd1=open("myfile.txt", O_RDONLY);
    close(2);
    printf("Donot enter anything:\n");
    scanf("%d", &n);
    /*
    Open a new terminal and type ls /proc/PID/fd before taking
    the input. The active fds of the process will be shown.
    */
    dup(fd1);
    printf("Donot enter anything:\n");
    /*
    Open a new terminal and type ls /proc/PID/fd before taking
    the input. The active fds of the process will be shown.
    */
    scanf("%d", &n);
    dup2(fd1,fd2);
    printf("Donot enter anything:\n");
    scanf("%d", &n);
    /*
    Open a new terminal and type ls /proc/PID/fd before taking
    the input. The active fds of the process will be shown.
    */
    return 0;
}
```

Observation**Specify: input & output**

13. Predict the output and check the descriptors in the process's **fd** directory using the command **ls /proc/PID/fd**. Write a conclusion about the execution style of the **dup2 ()** in contrast with **dup ()**.

```
int main() {
    int fd, fd1, fd2, fd3, dup2ret;
    char buf[12];
    printf("PID=%ld\n", (long) getpid());
    fd=open("read.c", O_RDONLY);
    fd1=open("read.c", O_RDONLY);
    fd2=open("select.c", O_RDONLY);
    close(fd1);
    dup2ret=dup2(fd, fd2);
    if(dup2ret== -1) {
        perror("dup2 error");
        return 1;
    }
    printf("Value of fd2=%d\n", fd2);
    printf("Dup2 return=%d\n", dup2ret);
    scanf("%d", &fd);
    return 0;
}
```

Observation	Output
•	
•	
•	
•	

14. Find the duplicate descriptors for the standard descriptors 0, 1, 2 after program becomes process. Also write the outputs of the given code snippet.

```
int main() {
    int i, fds[10];
    int fd=dup(0);
    printf("%d\n", fd);
    for(i=0; i<10; i++) {
        if(i%2==0)
            fds[i]=dup(1);
        else
            fds[i]=dup(2);
    }
    for(i=0; i<10; i++) {
        printf("%d\n", fds[i]);
    }
    while(1);
    return 0;
}
```

Observation	Output

15. Write the outputs of the given code snippet.

```
int main() {int fd, fd1, fd2, dup2ret;
    char buf[12];
    fd=open("read.c", O_RDONLY);
    fd1=open("read.c", O_RDONLY);
    dup2ret=dup2(fd, 19);
    if(dup2ret== -1) {
        perror("dup2 error");
        return 1;
    }
    printf("Dup2 return=%d\n", dup2ret);
    return 0; }
```

Observation	Output